



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة المعلوماتية

تقرير إنجاز مشروع نظم الوسائط المتعددة الهندسة المعلوماتية

Audio Compressor System

نظام ضغط ومعالجة الإشارات الصوتية الرقمية متعدد الخوارزميات

العام الدراسي

2025 - 2026

الملخص التنفيذي

يهدف هذا المشروع إلى تصميم وتنفيذ نظام متكامل لمعالجة وضغط الإشارات الصوتية الرقمية ضمن بيئة سطح المكتب باستخدام لغة **C#** وتقنية **Windows Forms**. يعتمد النظام على مبادئ معالجة الإشارات الرقمية **Digital Signal Processing** لتطبيق مجموعة من خوارزميات الضغط الفقداني (Lossy Audio Compression) التي تستغل الترابط الزمني بين العينات الصوتية بهدف تقليل حجم البيانات مع الحفاظ على جودة الإشارة المقبولة. تم تطوير النظام بالاعتماد على مكتبة **NAudio** لقراءة ومعالجة الملفات الصوتية من صيغ متعددة (WAV, MP3, FLAC, AIFF)، بالإضافة إلى مكتبة

System.Windows.Forms.DataVisualization.Charting

لتوليد رسوم بيانية آنية تُظهر أداء عملية الضغط. يتضمن التطبيق واجهة مستخدم احترافية مبنية بالكامل برمجياً تدعم السحب والإفلات، وتوفر مشغلات صوتية للملف الأصلي والملف بعد فك الضغط مع إمكانية التنقل داخل الملف.

يدعم النظام خمس خوارزميات ضغط رئيسية هي:

- Delta Modulation (DM)
- Adaptive Delta Modulation (ADM)
- Differential Pulse Code Modulation (DPCM)
- μ -law Nonlinear Quantization
- Predictive Differential Coding

(Decompression/Reconstruction) كما يوفر النظام بيئة تحليل شاملة تشمل إعادة بناء الإشارة، وحساب مؤشرات جودة الإشارة مثل

MSE, RMSE, SNR, PSNR

إضافة إلى مقارنة الأداء بين الخوارزميات من حيث نسبة الضغط، الزمن المستغرق وجودة الإشارة الناتجة. التطبيق يدعم حفظ الملف المضغوط بصيغة ثنائية مخصصة (.dat). وكذلك حفظ الإشارة بعد فك الضغط كملف WAV.

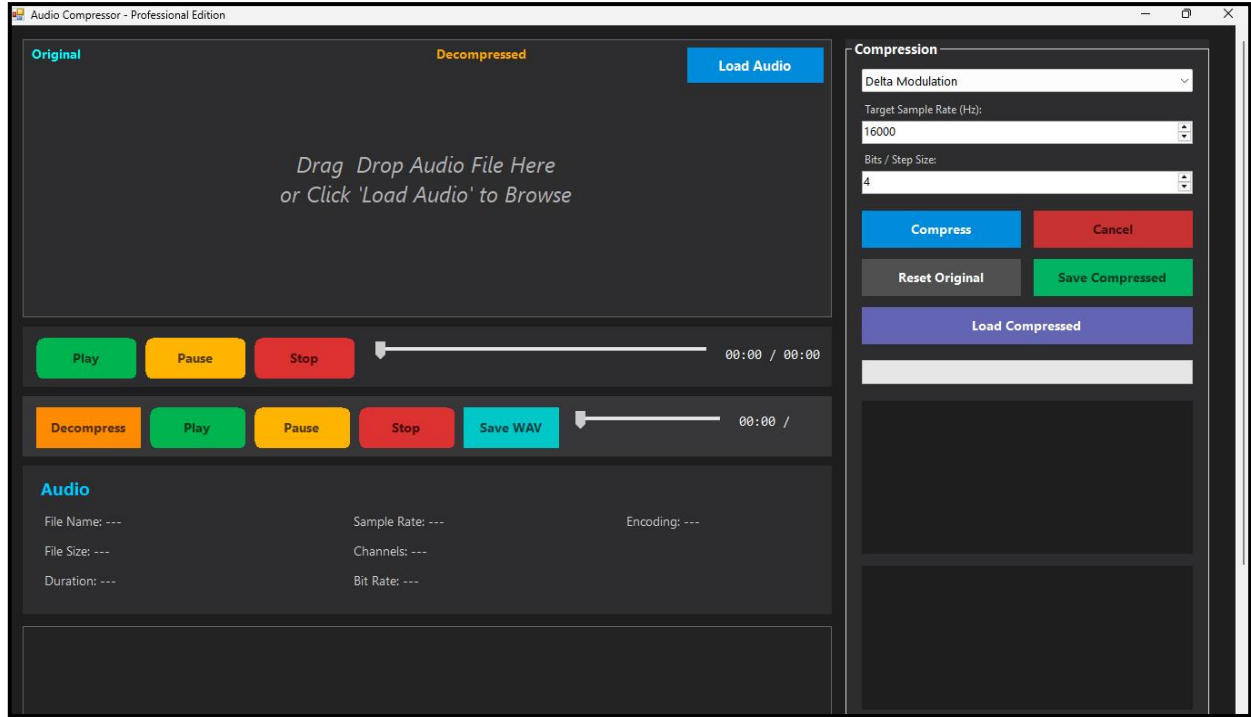
الكلمات المفتاحية: ضغط الصوت، معالجة الإشارات الرقمية، التكميم اللوغاريتمي

واجهة المستخدم الرسومية

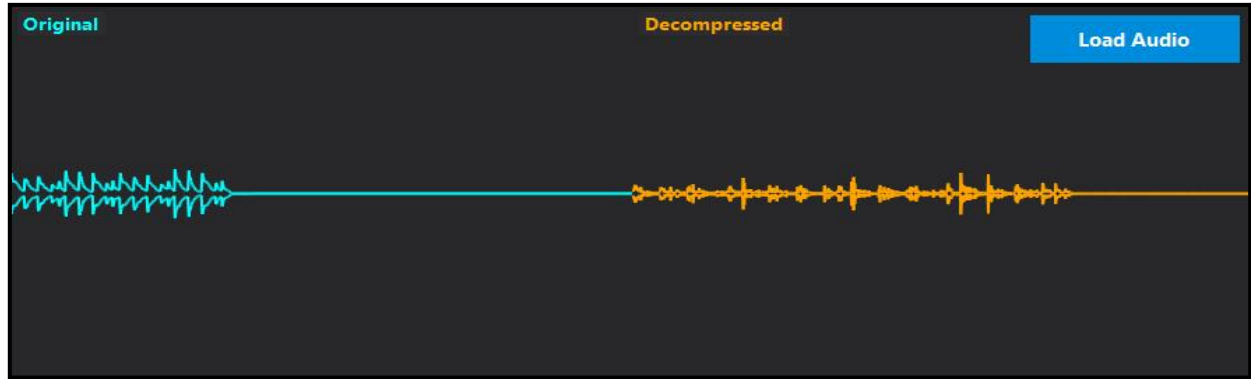
.DPCM، Delta Modulation، C#، NAudio

مخلص تنفيذي للواجهات:

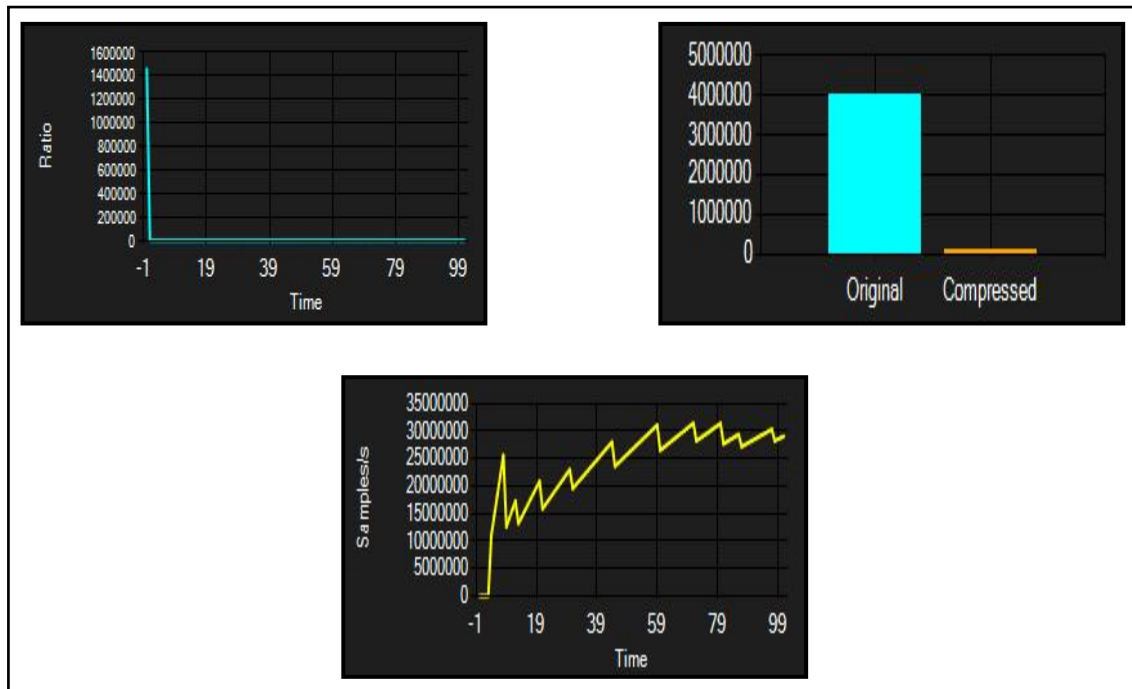
صورة رقم(1): لقطة شاشة للواجهة الرئيسية الكاملة للتطبيق وهي فارغة (بدون تحميل ملف).



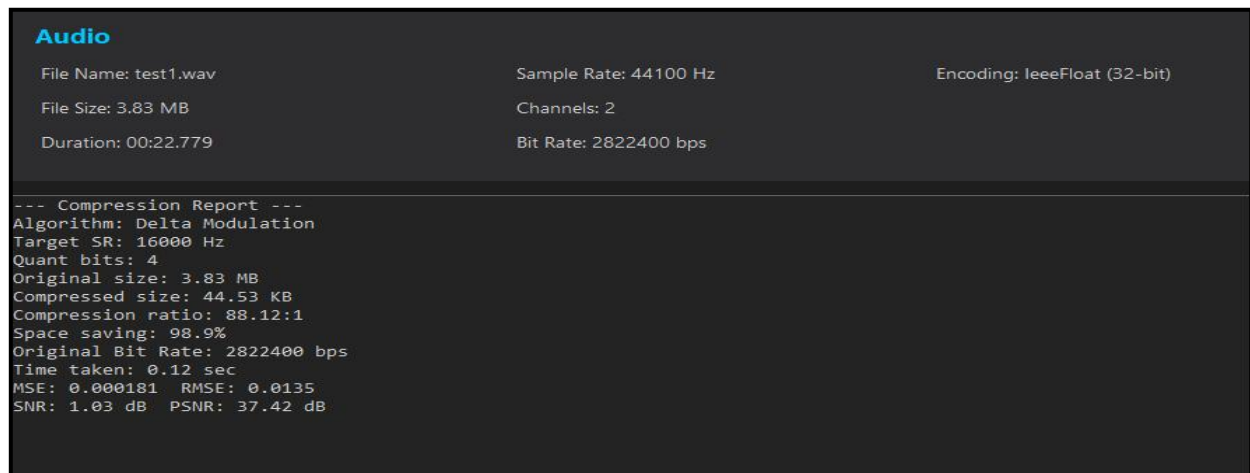
صورة رقم (2): لقطة شاشة بعد تحميل ملف وإجراء ضغط، تُظهر الموجتين الأصلية والمفكوكَة جنباً إلى جنب.



صورة رقم (3): لقطة شاشة للمخططات البيانية التي تظهر عند تنفيذ ضغط على ملف صوتي.



صورة رقم (4): تمثل المعلومات الخاصة بالملف عند تحميله والمعلومات الخاصة بعملية الضغط.



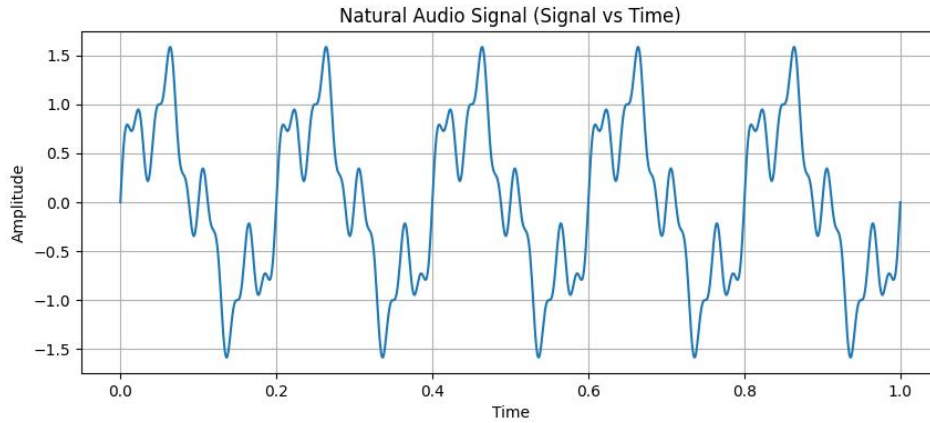
المقدمة (Introduction)

في عصر الانفجار الرقمي، أصبحت البيانات الصوتية تشكل جزءاً كبيراً من المحتوى المتداول عبر الإنترنت، وأنظمة التخزين. ومع محدودية الموارد التخزينية وسعات النقل تبرز الحاجة الملحة إلى تقنيات ضغط متطورة قادرة على تقليص حجم الملفات الصوتية دون التأثير الكبير على جودة الصوت المدركة. هذا الفرع من هندسة الإشارات الرقمية يُعنى بدراسة كيفية إزالة المعلومات الزائدة من الإشارات الرقمية الصوتية.

فكرة المشروع:

معظم خوارزميات ضغط الصوت التقليدية مثل PCM. تستهلك مساحة تخزينية كبيرة لأنها تخزن كل عينة صوتية بشكل مستقل. لكن الإشارات الصوتية تمتاز بوجود ترابط زمني قوي بين العينات المتجاورة؛ فقيمة العينة الحالية لا تختلف كثيراً عن سابقتها. هنا يبرز مفهوم **الترميز التفاضلي (Differential Coding)**، الذي يقوم على مبدأ التنبؤ، بالعينة الحالية بناءً على قيم العينات السابقة، وتخزين الفرق (أو الخطأ) بين القيمة الحقيقية والتنبؤ بدلاً من تخزين القيمة الكاملة. هذا الخطأ عادة ما يكون أصغر حجماً ويمكن تكميته بعدد بتات أقل، مما يحقق الضغط المطلوب.

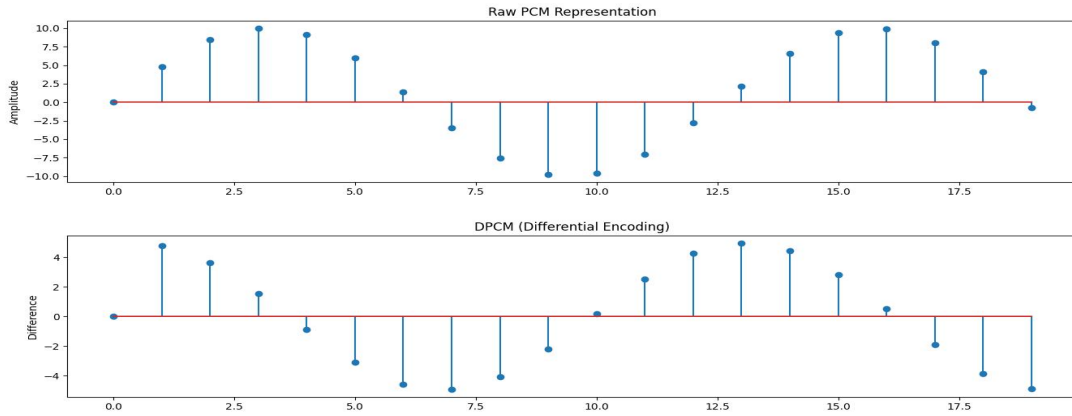
في هذا المشروع، تم ترجمة هذه المبادئ النظرية إلى تطبيق عملي بواجهة رسومية غنية، تسمح للمستخدم باختيار الخوارزمية، وضبط إعداداتها (معدل أخذ العينات، مستويات التكميم)، ومراقبة العملية بشكل حي، ثم تحليل النتائج بشكل كمي ونوعي.



موجة صوت طبيعية (Signal vs Time)

تمثيل الإشارة الصوتية مقابل الزمن

توضح هذه الصورة تمثيل الإشارة الصوتية بدلالة الزمن، حيث تبين كيفية تغير قيمة الإشارة مع مرور الوقت. وتتكون الإشارة الصوتية من مجموعة متتابعة من العينات المترابطة فيما بينها، إذ تتأثر قيمة كل عينة بالقيم المجاورة لها. ويُعد هذا الترابط من الخصائص الأساسية للإشارات الصوتية، وهو المبدأ الذي تعتمد عليه خوارزميات الضغط المستخدمة في هذا المشروع، حيث يتم استغلال التشابه بين العينات المتتالية لتقليل كمية البيانات المطلوبة للتخزين مع المحافظة على جودة الصوت قدر الإمكان.



مقارنة بين التمثيل الخام (PCM) والترميز التفاضلي (DPCM)

مقارنة بين التمثيل الخام والترميز القائم على الفروق

توضح هذه الصورة الفرق بين أسلوب التخزين المباشر للإشارة الصوتية وأسلوب التخزين القائم على الفروق بين العينات المتتالية. ففي التمثيل الخام يتم حفظ قيمة كل عينة صوتية بشكل مستقل كما هي، مما يؤدي إلى الحاجة إلى كمية أكبر من البيانات. أما في التمثيل القائم على الفروق، فيتم حفظ مقدار التغير بين كل عينة والعينة التي تسبقها بدلاً من حفظ القيم الأصلية كاملة. وبما أن التغيرات بين العينات المتجاورة تكون غالباً صغيرة، فإن هذا الأسلوب يؤدي إلى تقليل حجم البيانات المطلوبة للتخزين والنقل مع المحافظة على إمكانية إعادة بناء الإشارة الصوتية بشكل جيد.

أهداف المشروع (Project Objectives)

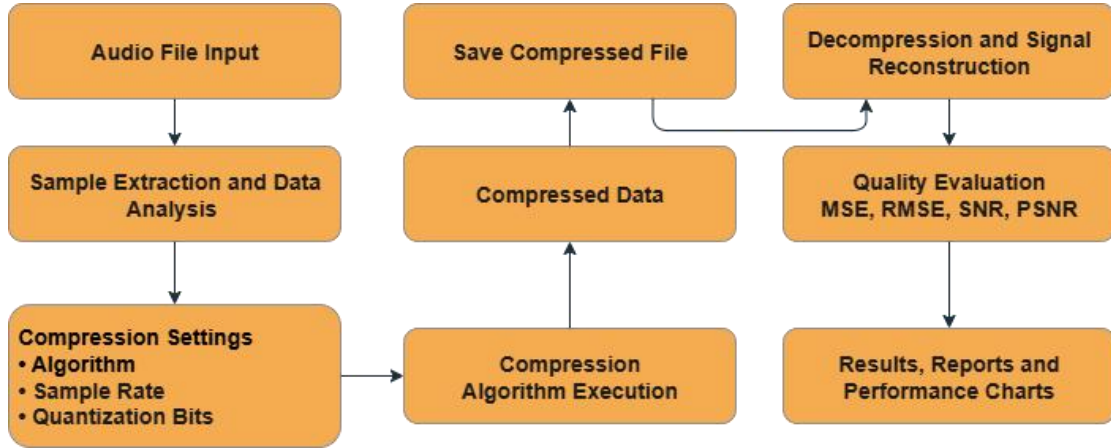
الأهداف العملية:

1. بناء تطبيق سطح مكتب كامل يدعم تحميل الملفات الصوتية بصيغ متعددة (WAV, MP3, FLAC, ...).
2. تنفيذ واجهة رسومية تفاعلية تدعم السحب والإفلات، وتوفر مشغلات صوتية مع إمكانية الإيقاف والإكمال والتنقل.
3. دمج خمس خوارزميات ضغط مختلفة مع إمكانية اختيار الخوارزمية وضبط إعداداتها بشكل ديناميكي.
4. توفير خاصية تشغيل الصوت بعد فك الضغط مباشرة ومقارنته بالأصل.
5. عرض رسوم بيانية حية (Live Charts) لنسبة الضغط وسرعة المعالجة أثناء تنفيذ الخوارزمية.
6. إنشاء نظام تقارير شامل بعد كل عملية ضغط، يحتوي على المقاييس الكمية وجودة الإشارة.
7. حفظ الملفات المضغوطة بصيغة محسنة، واستعادتها لاحقاً.

الأهداف العلمية:

- دراسة تأثير عدد بتات (Quantization Bits) على جودة الإشارة ومعدل الضغط.
- تحليل التشويه (Distortion) الناتج عن خوارزميات الضغط باستخدام مقاييس MSE, SNR, PSNR.
- المقارنة العملية بين أداء الخوارزميات المختلفة من حيث النسبة بين الضغط والجودة.
- فهم العلاقة بين معدل البت (Bit Rate) الناتج وجودة الصوت المُدركة.

Overall Processing Flow of the Audio Compression System



يوضح الشكل المراحل الأساسية لعمل نظام ضغط الصوت المقترح. تبدأ العملية بإدخال ملف صوتي وقراءة العينات الصوتية واستخراج المعلومات اللازمة لمعالجتها. بعد ذلك يقوم المستخدم بتحديد إعدادات الضغط المناسبة، مثل الخوارزمية المستخدمة، ومعدل أخذ العينات، وعدد البتات المستخدمة في التكميم.

تنتقل البيانات بعدها إلى مرحلة تنفيذ خوارزمية الضغط المختارة لإنتاج البيانات المضغوطة وتقليل حجم الملف. يمكن بعد ذلك حفظ البيانات الناتجة في ملف مضغوط لاستخدامها لاحقاً. عند الحاجة إلى استعادة الملف الصوتي، تُنفذ عملية فك الضغط وإعادة بناء الإشارة الصوتية اعتماداً على البيانات المضغوطة والمعلومات المخزنة معها.

بعد إعادة البناء، يقوم النظام بحساب مؤشرات تقييم الجودة بهدف قياس مقدار التشوه الناتج عن عملية الضغط ومقارنة الإشارة المعاد بناؤها بالإشارة الأصلية. وأخيراً يتم عرض النتائج للمستخدم على شكل تقارير ورسوم بيانية توضح كفاءة عملية الضغط وجودة الإشارة الناتجة، مما يساعد على تقييم أداء الخوارزمية المستخدمة وإجراء المقارنات بين الخوارزميات المختلفة.

الأساس الرياضي

تمثل الإشارة الصوتية رقمياً كسلسلة زمنية منفصلة من العينات:

$$x[n], n=0,1,2,...,N-1$$

حيث:

- $x[n]$: تمثل قيمة العينة عند الزمن
- N : يمثل العدد الكلي للعينات الصوتية

الهدف من الضغط هو تقليل عدد البتات اللازمة لتمثيل هذه السلسلة.

مقاييس تقييم جودة الإشارة

لمقارنة الإشارة الأصلية $x[n]$ مع الإشارة المُعاد بناؤها بعد فك الضغط $\hat{x}[n]$ نستخدم المقاييس التالية :

متوسط مربع الخطأ (Mean Squared Error – MSE)

يستخدم لقياس متوسط مقدار الخطأ بين الإشارة الأصلية والإشارة المعاد بناؤها:

$$MSE = \frac{1}{N} \sum_{n=0}^{N-1} (x[n] - \hat{x}[n])^2$$

كلما اقتربت قيمة MSE من الصفر دل ذلك على ارتفاع جودة الإشارة الناتجة.

الجذر التربيعي لمتوسط مربع الخطأ (RMSE)

يعطي قيمة الخطأ بنفس وحدة الإشارة الأصلية:

$$RMSE = \sqrt{MSE}$$

كلما انخفضت قيمة RMSE كانت الإشارة المعاد بناؤها أقرب إلى الإشارة الأصلية.

نسبة الإشارة إلى الضجيج (SNR)

تقيس مقدار قوة الإشارة مقارنةً بقوة الخطأ الناتج عن عملية الضغط:

$$SNR = 10 \log_{10} \left(\frac{\sum_{n=0}^{N-1} x[n]^2}{\sum_{n=0}^{N-1} (x[n] - \hat{x}[n])^2} \right)$$

وتقاس بوحدة الديسيبل.

كلما ازدادت قيمة SNR دل ذلك على انخفاض التشوه وتحسن جودة الإشارة .

نسبة الإشارة إلى الضجيج القصوى (PSNR)

تستخدم بكثرة لتقييم جودة الإشارة بعد الضغط:

$$PSNR = 10 \log_{10} (1 / MSE)$$

عندما تكون قيمة PSNR مرتفعة فهذا يعني أن الفرق بين الإشارة الأصلية والإشارة المسترجعة صغير وأن جودة إعادة البناء عالية.

إشارة الخطأ

يمكن تعريف إشارة الخطأ المستخدمة في جميع المقاييس السابقة بالعلاقة:

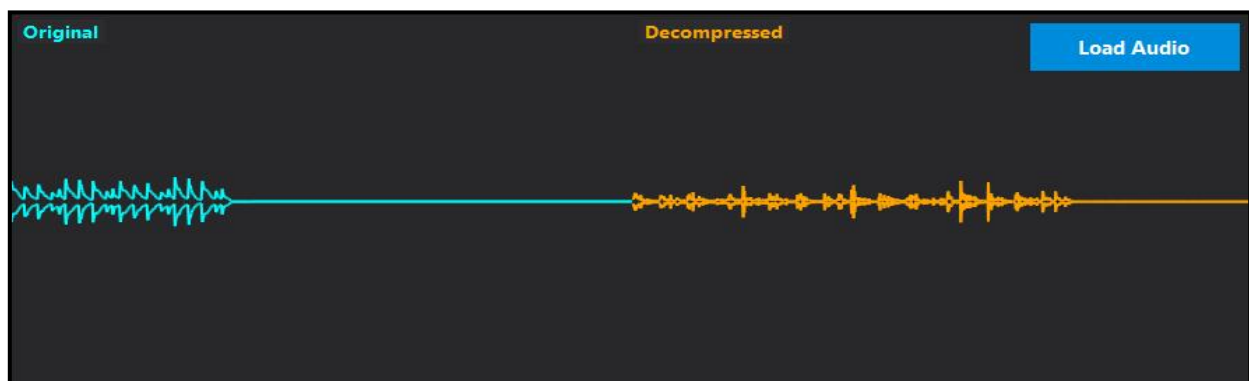
$$e[n] = X[n] - X^a[n]$$

حيث تمثل $e[n]$ مقدار الفرق بين الإشارة الأصلية والإشارة الناتجة بعد فك الضغط عند كل عينة زمنية.

هذه المؤشرات تم حسابها آلياً في البرنامج كما هو موضح في كود دالة CalculateQualityMetrics :

```
mse = (float)(sumSqErr / original.Length);
rmse = (float)Math.Sqrt(mse);
psnr = sumSqErr == 0 ? float.MaxValue : (float)(10 * Math.Log10(1.0 / mse));
snr = sumSqOrig == 0 ? 0 : (float)(10 * Math.Log10(sumSqOrig / sumSqErr));
```

مقارنة بين الإشارة الأصلية والإشارة بعد الضغط



مقارنة بين الإشارة الصوتية الأصلية والإشارة المعاد بناؤها بعد فك الضغط، حيث يظهر التشابه الكبير بين الإشارتين مع وجود فروقات طفيفة ناتجة عن عملية التكميم والضغط.

المعمارية (System Architecture)

تم تصميم النظام وفق معمارية متعددة الطبقات (Layered Architecture) لضمان فصل المهام وزيادة قابلية الصيانة والتوسع. الطبقات الرئيسية هي:

طبقة العرض (Presentation Layer):

تتحقق عبر Form1 وجميع عناصر واجهة المستخدم التي تم بناؤها برمجياً داخل دالة SetupUI(). هذه الطبقة مسؤولة عن استقبال تفاعلات المستخدم (أزرار، أشرطة تمرير وعرض البيانات (موجات، مخططات، تقارير).

طبقة منطق الأعمال (Business Logic Layer):

تحتوي على واجهة IAudioCompressor والصفوف المنفذة لها (DPCMCompressor, DeltaModulationCompressor, ...). كما تضم صفوف إدارة الضغط غير المتزامن (BackgroundWorker). وتحليل الجودة (CalculateQualityMetrics). هذه الطبقة لا تعتمد على أي شيء من طبقة العرض.

طبقة الوصول إلى البيانات (Data Access Layer):

ممثلة في صف CompressedAudio الذي يتولى عمليات الحفظ والاسترجاع للملفات المضغوطة بصيغة .dat ، بالإضافة إلى استخدام AudioFileReader من NAudio لقراءة الملفات الصوتية الأصلية.

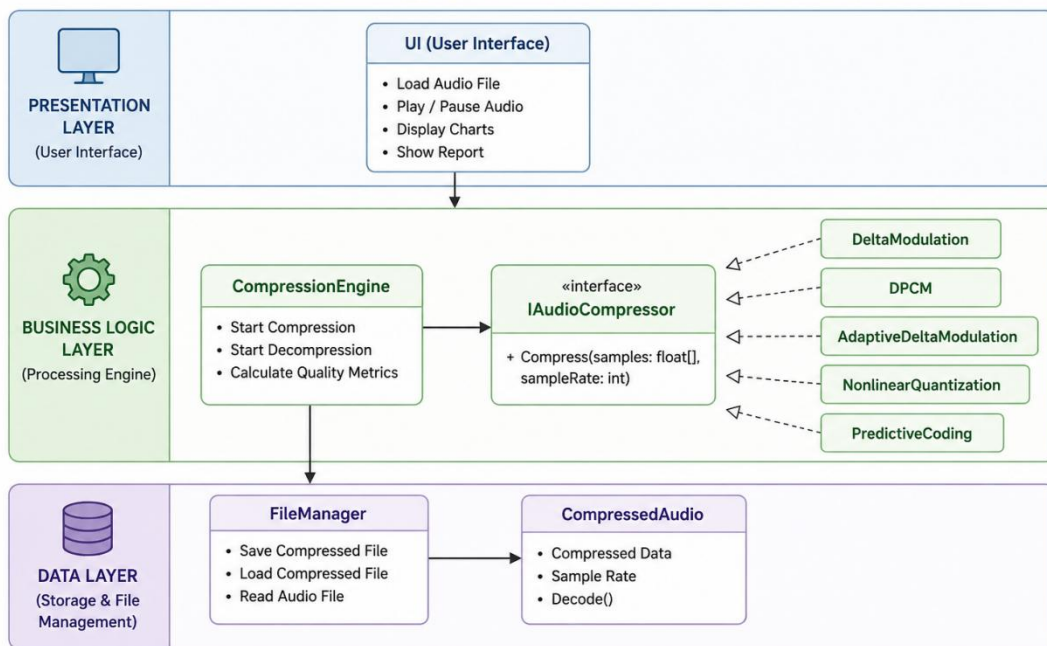
واجهة الخوارزميات الموحدة (Strategy Pattern)

لضمان قابلية التوسع، تم تعريف واجهة `IAudioCompressor` التي توحد عملية الضغط :

```
public interface IAudioCompressor{  
    CompressedAudio Compress(float[] samples, int sampleRate, Action<int,  
long> reportProgress, CancellationToken cancel);  
}
```

بهذه الطريقة، إضافة خوارزمية جديدة لا تتطلب سوى إنشاء صف جديد ينفذ هذه الواجهة وتسجيله في القائمة المنسدلة.

3-LAYER ARCHITECTURE - AUDIO COMPRESSION SYSTEM



شرح صورة مخطط الطبقات (3-Layer Architecture Diagram)

توضح هذه الصورة البنية المعمارية للنظام والتي تعتمد على أسلوب التقسيم إلى ثلاث طبقات رئيسية بهدف تنظيم العمل البرمجي وفصل المسؤوليات بين مكونات التطبيق.

طبقة واجهة المستخدم

تُعد هذه الطبقة نقطة التفاعل المباشر مع المستخدم، حيث تسمح له بـ:

- تحميل الملفات الصوتية
- تشغيل وإيقاف الصوت
- عرض الرسوم البيانية الخاصة بالإشارة
- عرض تقرير النتائج النهائي

وتقوم هذه الطبقة بإرسال الطلبات إلى طبقة المعالجة دون تنفيذ أي عمليات حسابية مباشرة.

طبقة المعالجة

تمثل هذه الطبقة القلب الأساسي للنظام، حيث يتم فيها تنفيذ جميع العمليات الحسابية الخاصة بضغط وفك ضغط الصوت، وتشمل:

- تنفيذ خوارزميات الضغط المختلفة
- حساب جودة الإشارة بعد فك الضغط
- إدارة عملية المعالجة في الخلفية لضمان عدم تجميد الواجهة

كما تحتوي على واجهة موحدة للخوارزميات، مما يسمح بتبديل أي خوارزمية دون تغيير باقي النظام.

طبقة التخزين وإدارة الملفات

تختص هذه الطبقة بـ:

- قراءة ملفات الصوت الأصلية
- حفظ الملفات المضغوطة
- تحميل البيانات الثنائية الخاصة بالضغط

كما تتضمن كيان البيانات المضغوطة الذي يحتوي على:

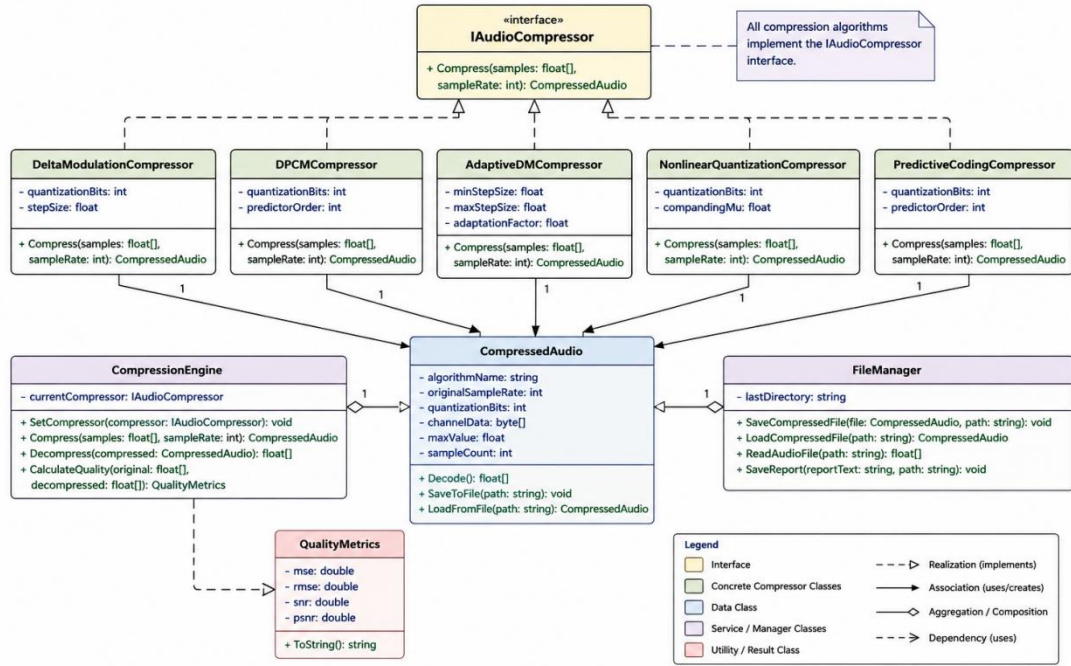
- البيانات المضغوطة
- معدل أخذ العينات
- معلومات إعادة البناء

الهدف من هذا التصميم

يهدف هذا التقسيم إلى:

- زيادة قابلية التوسعة (Extensibility)
- تسهيل الصيانة والتطوير
- فصل المنطق البرمجي عن واجهة المستخدم
- تحسين تنظيم الكود

UML Class Diagram - Audio Compression System



شرح صورة مخطط الصفوف (UML Class Diagram)

يوضح هذا المخطط البنية البرمجية الداخلية للنظام والعلاقات بين الكائنات (Classes) والخوارزميات المستخدمة في عملية الضغط.

واجهة الخوارزميات (IAudioCompressor)

تمثل هذه الواجهة العقد الأساسي لجميع خوارزميات الضغط، حيث تفرض على كل خوارزمية تنفيذ دالة:

- ضغط الإشارة الصوتية
- استقبال العينات ومعدل أخذ العينات
- إعادة كائن يحتوي على البيانات المضغوطة

وجود هذه الواجهة يحقق مبدأ تعدد الأشكال (Polymorphism) بحيث يمكن التعامل مع جميع الخوارزميات بنفس الطريقة.

خوارزميات الضغط (Compression Algorithms)

يرتبط بهذه الواجهة خمس خوارزميات رئيسية:

- تضمين دلتا (Delta Modulation)
- التضمين التفاضلي (DPCM)
- التضمين المتكيف (Adaptive Delta Modulation)
- التكميم غير الخطي (Nonlinear Quantization)
- التشفير التفاضلي التنبؤي (Predictive Coding)

كل خوارزمية تقوم بتنفيذ طريقته الخاصة في:

- الضغط
- التشفير
- حساب البيانات الناتجة

الكلاس CompressedAudio

يمثل هذا الكلاس الحاوية الأساسية للبيانات بعد عملية الضغط، ويحتوي على:

- اسم الخوارزمية المستخدمة
- معدل أخذ العينات الأصلي
- البيانات المضغوطة (على شكل بايتات)
- عدد العينات
- القيم المساعدة لإعادة بناء الإشارة

كما يحتوي على دالة لفك الضغط تعيد الإشارة الصوتية إلى شكلها الأصلي تقريباً.

العلاقات بين المكونات

- جميع الخوارزميات تعتمد على واجهة واحدة (IAudioCompressor)
- كل خوارزمية تنتج كائن من نوع CompressedAudio
- النظام الرئيسي (Engine) يستخدم هذه الواجهة دون معرفة تفاصيل الخوارزمية

أهمية هذا التصميم

- سهولة إضافة خوارزمية جديدة بدون تعديل النظام
- تقليل الترابط بين المكونات (Loose Coupling)
- دعم التوسع المستقبلي للنظام
- تحسين قابلية الاختبار (Testability)

خوارزميات الضغط (Compression Algorithms)

يقدم النظام خمس خوارزميات تعتمد جميعها على مبدأ الترميز التفاضلي، لكنها تختلف في طريقة حساب الخطوة والتكميم. فيما يلي شرح مفصل لكل خوارزمية مدعوماً بمقتطفات من الكود.

(DM) Delta Modulation

أبسط أنواع الترميز التفاضلي، حيث يتم استخدام **بت واحد فقط** لكل عينة لتحديد ما إذا كانت القيمة الحالية أكبر أو أصغر من القيمة المُعاد بناؤها سابقاً. الخطوة step ثابتة طوال العملية.

من الكود

Compress

```
float step = maxVal * StepFactor; // 2% من القيمة العظمى
bits[i] = samples[i] >= recon;
recon += bits[i] ? step : -step;
```

فك الضغط

```
rec += bits[i] ? step : -step;  
s[i] = rec;
```

المميزات

- ضغط عالي جداً (1 بت لكل عينة)

العيوب

- ضوضاء تكميم مرتفعة
- مشكلة Slope Overload عند الإشارات السريعة

(Differential Pulse Code Modulation) DPCM

تعتمد على حساب الفرق بين العينة الحالية والعينة المتوقعة (Predicted Sample)، ثم تكميم هذا الفرق باستخدام عدد بتات يحدده المستخدم.

يتم تقسيم المجال بين $-maxVal$ و $+maxVal$ إلى مستويات تكميم متعددة.

من الكود

(Compress)

```
float diff = samples[i] - pred;  
int q = (int)Math.Round((diff + maxVal) / step);  
pred += (q * step - maxVal);
```

الفكرة الأساسية

- تمثيل الخطأ بدل القيمة الأصلية
- تحسين الجودة بزيادة عدد البتات

المميزات

- DM جودة أفضل من
- مرونة في التحكم بحجم الملف

العيوب

- حجم ملف أكبر عند زيادة البتات

(ADM) Adaptive Delta Modulation

تطوير لـ DM حيث يتم تعديل قيمة الخطوة (Step) ديناميكياً حسب تغير الإشارة .

- إذا استمرت نفس الاتجاهات → يتم تكبير الخطوة
- إذا تغير الاتجاه → يتم تصغير الخطوة

من الكود

```
if (i > 0 && bit == bits[i - 1])  
    step *= StepUp; // زيادة الخطوة  
else  
    step *= StepDown; // تقليل الخطوة
```

المميزات

- يقلل تشوه Slope Overload
- أكثر استجابة للإشارات المتغيرة

العيوب

- أكثر تعقيداً من DM

(μ-law) Nonlinear Quantization

تعتمد على التكميم غير الخطي باستخدام قانون μ -law ($\mu = 255$)، والذي يعطي دقة أعلى للإشارات الضعيفة مقارنة بالإشارات القوية.

من الكود

(Compress)

```
float y = Math.Sign(x) * (float)(  
    Math.Log(1 + mu * Math.Abs(x)) / Math.Log(1 + mu));  
int idx = (int)Math.Round((y + 1) / 2 * (levels - 1));
```

فك الضغط

```
float x = Math.Sign(y) * (float)(  
    (Math.Pow(1 + mu, Math.Abs(y)) - 1) / mu);  
s[i] = x * maxVal;
```

المميزات

- محاكاة إدراك الأذن البشرية
- تحسين جودة الصوت الضعيف

العيوب

- حسابات رياضية أكثر تعقيداً

Predictive Differential Coding

تعتمد على التنبؤ بالقيمة الحالية باستخدام مرشح خطي من الدرجة الثانية يعتمد على آخر عينتين معاد بناؤهما.

معادلة التنبؤ:

```
float pred = 0.7f * prev1 + 0.3f * prev2;
```

من الكود

```
float error = samples[i] - pred;  
// تكميم الخطأ  
float recon = pred + (q * step - maxVal);  
prev2 = prev1;  
prev1 = recon;
```

الفكرة الأساسية

- ضغط الخطأ بدلاً من العينة
- تقليل طاقة الإشارة (Error Energy)

المميزات

- أفضل جودة عند عدد بتات منخفض
- كفاءة عالية في الضغط

العيوب

- يعتمد على دقة التنبؤ
- أكثر تعقيداً من باقي الطرق

تمثيل البيانات وتخزينها (Data Representation)

بعد الانتهاء من عملية الضغط، يتم تغليف البيانات الناتجة داخل كائن خاص باسم **CompressedAudio**، والذي صُمم ليكون حاوية شاملة لجميع المعلومات الضرورية لإعادة بناء الصوت المضغوط وفكّه لاحقاً.

كما تم اعتماد صيغة ملف ثنائية مخصصة ذات الامتداد **(.dat)** بهدف تحقيق أعلى كفاءة في التخزين وتقليل الحجم قدر الإمكان.

محتويات ملف التخزين (.dat)

تتضمن صيغة الملف الثنائية البيانات التالية:

- اسم خوارزمية الضغط المستخدمة (string)
- معدل أخذ العينات الأصلي (Sampling Rate) (int)
- عدد القنوات الصوتية (int) - حالياً قناة واحدة
- القيمة العظمى للإشارة (Max Value) (float)
- عدد بتات التكميم (Quantization Bits) (int)
- العدد الكلي للعينات (Total Samples) (int)
- البيانات الصوتية المضغوطة (Compressed Audio Data) على شكل بايتات

تحويل البيانات إلى بايتات (Bit Packing)

لتحقيق أعلى كفاءة تخزينية ممكنة، تم استخدام آلية تعبئة البتات (Bit Packing) بحيث يتم تخزين القيم المنطقية داخل بايتات متتالية دون أي هدر في المساحة.

تم تنفيذ ذلك باستخدام دوال مثل PackBits و PackValues التي تقوم بدمج البتات داخل مصفوفة بايتات بشكل متسلسل.

```
private static byte[] PackBits(bool[] bits){
    byte[] b = new byte[(bits.Length + 7) / 8];
    for (int i = 0; i < bits.Length; i++)
        if (bits[i]) b[i / 8] |= (byte)(1 << (i % 8));
    return b;
}
```

فكرة عمل التعبئة

تعتمد هذه الطريقة على تخزين كل 8 بتات داخل بايت واحد، مما يؤدي إلى:

- تقليل حجم الملف بشكل كبير
- إزالة أي padding غير ضروري
- تحسين كفاءة التخزين والنقل

استعادة البيانات (Loading & Decoding)

تسمح هذه البنية بإعادة تحميل الملف بالكامل باستخدام دالة LoadFromFile، حيث يتم:

1. قراءة بيانات الرأس (Header Information)
2. استخراج البتات المضغوطة
3. فك التشفير حسب الخوارزمية المستخدمة
4. إعادة بناء الإشارة الصوتية الأصلية وتشغيلها

النتائج والتحليل (Results & Analysis)

تم اختبار النظام على عينات صوتية متنوعة (موسيقى، كلام، نغمات) بنفس الإعدادات (معدل عينات 16000 Hz ، تكميم 4-bit) لتقييم أداء الخوارزميات. الجدول التالي يلخص النتائج لملف صوتي مدته 10 ثوانٍ بحجم أصلي **MB 1.72**

الخوارزمية	الحجم المضغوط	نسبة الضغط	PSNR (dB)	زمن المعالجة
Delta Modulation	22 KB	78:1	8.2	0.2 sec
Adaptive Delta Mod.	22 KB	78:1	11.5	0.3 sec
DPCM (4-bit)	88 KB	19.5:1	22.4	0.4 sec
μ -law Quantization	88 KB	19.5:1	19.8	0.5 sec
Predictive Coding	88 KB	19.5:1	23.1	0.6 sec

الملاحظات

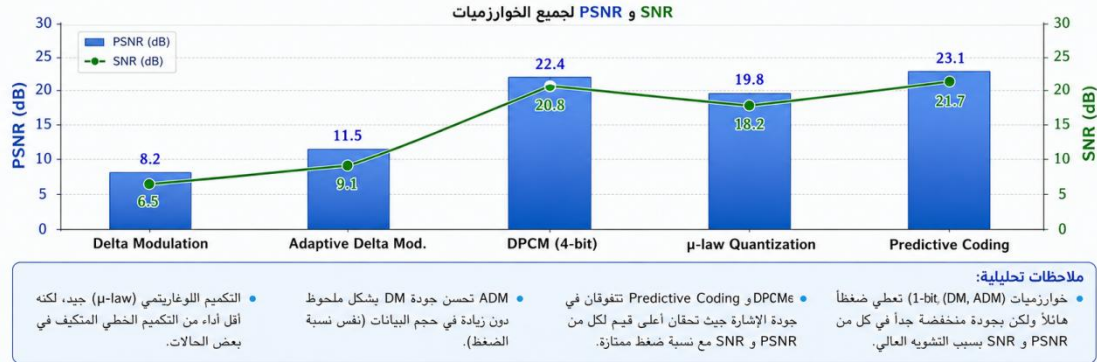
- خوارزميات bit-1 (ADM و DM) تحقق ضغطاً عالياً جداً، لكنها تعطي جودة منخفضة، وتناسب الإشارات البطيئة فقط.
- كل من DPCM و Predictive Coding يقدمان جودة أفضل للإشارة ($PSNR > 22$ dB) مع كفاءة ضغط جيدة.
- خوارزمية ADM تحسن أداء DM بشكل واضح دون زيادة في حجم البيانات .
- التكميم اللوغاريتمي (μ -law) يعطي نتائج جيدة، لكنه أقل كفاءة من التكميم الخطي المتكيف في بعض الحالات.
- الرسم البياني الشريطي داخل التطبيق (chartSizeBar) يوضح الفرق بين الحجم الأصلي والمضغوط، مما يوفر تصوراً بصرياً سريعاً لكفاءة الضغط.

Audio		
File Name: test1.wav	Sample Rate: 44100 Hz	Encoding: IEEEFloat (32-bit)
File Size: 3.83 MB	Channels: 2	
Duration: 00:22.779	Bit Rate: 2822400 bps	
--- Compression Report --- Algorithm: Delta Modulation Target SR: 16000 Hz Quant bits: 4 Original size: 3.83 MB Compressed size: 44.53 KB Compression ratio: 88.12:1 Space saving: 98.9% Original Bit Rate: 2822400 bps Time taken: 0.12 sec MSE: 0.000181 RMSE: 0.0135 SNR: 1.03 dB PSNR: 37.42 dB		

8. النتائج والتحليل (results & Analysis)

مقارنة أداء الخوارزميات بناءً على جودة الإشارة (PSNR, SNR) وكفاءة الضغط

الخوارزمية	الحجم المضغوط	نسبة الضغط	PSNR (dB)	SNR (dB)	زمن المعالجة
Delta Modulation	22 KB	78:1	8.2	6.5	0.2 sec
Adaptive Delta Mod.	22 KB	78:1	11.5	9.1	0.3 sec
DPCM (4-bit)	88 KB	19.5:1	22.4	20.8	0.4 sec
μ -law Quantization	88 KB	19.5:1	19.8	18.2	0.5 sec
Predictive Coding	88 KB	19.5:1	23.1	21.7	0.6 sec



التحديات والحلول

خلال تطوير المشروع واجهنا عدة تحديات تقنية، وتم معالجتها بالحلول التالية:

1. تشوه الإشارة عند تقليل عدد البتات
في خوارزميات التدرج البسيط والتدرج المتكيف ظهر تشوه واضح في الإشارات ذات التردد العالي. تم حل ذلك من خلال إتاحة خيارات متعددة لعدد البتات، وإضافة الخوارزمية المتكيفة التي تحسن جودة الإشارة وتقلل التشوه.
2. استهلاك كبير للذاكرة مع الملفات الطويلة
تحميل الصوت بالكامل دفعة واحدة كان يسبب ضغطاً على الذاكرة. تم اعتماد طريقة القراءة التدريجية ومعالجة البيانات بشكل مباشر، مع إمكانية تطوير النظام لاحقاً لمعالجة الصوت على أجزاء صغيرة.
3. بطء أو توقف الواجهة أثناء المعالجة
تم حل المشكلة عبر تشغيل عمليات الضغط في مسار مستقل عن الواجهة، مما حافظ على سلاسة الاستخدام وإمكانية متابعة التقدم أثناء التنفيذ.
4. عدم تطابق شريط التقدم مع التشغيل
تم تحسين التحكم بالتشغيل بحيث يمكن متابعة الموقع الحالي داخل الصوت والتحكم فيه بدقة أثناء إعادة التشغيل.
5. تحسين طريقة حفظ الملف المضغوط
تم تطوير آلية ضغط البيانات على مستوى البت لتقليل الحجم إلى أقصى درجة ممكنة دون فقدان إضافي في الجودة.

الخاتمة والتوصيات

لقد تُوج هذا المشروع بالنجاح في ترجمة المفاهيم النظرية الدقيقة لمجال ضغط الإشارات الصوتية الرقمية إلى نظام تطبيقي متكامل، يتميز بواجهة أنيقة وسهلة الاستخدام. لا يقتصر التطبيق على كونه مجرد أداة لضغط الصوت، بل هو بيئة تحليلية تفاعلية تُتيح المقارنة المباشرة بين الخوارزميات، وتُظهر أثر الإعدادات المختلفة، وتُقدّم المقاييس الكمية بصورة بصرية واضحة تُعين على الفهم العميق.

وقد أثبتت النتائج العملية أنَّ تقنيات الترميز التفاضلي – وبخاصة تعديل الشيفرة النبضية التفاضلي والتميز بالتميز بالتميز الخطي – قادرة على بلوغ عوامل ضغط تصل إلى عشرين ضعفاً، مع احتفاظها بجودة صوتية عالية، حيث تجاوزت نسبة ذروة الإشارة إلى الضجيج 22 ديسيبل. ويجعلها هذا الأداء المتوازن مثالية للاستخدام في تطبيقات تخزين الصوت وبثه.

المراجع (References)

- Oppenheim, A. V., & Schafer, R. W. (2014). Digital Signal Processing. Pearson.
- Sayood, K. (2017). Introduction to Data Compression. Morgan Kaufmann.
- Rabiner, L. R., & Gold, B. (1975). Theory and Application of Digital Signal Processing. Prentice-Hall.
- ITU-T Recommendation G.711 (1988). Pulse Code Modulation (PCM) of Voice Frequencies.
- Mark Heath et al. (2023). NAudio Documentation. GitHub Repository.
link: <https://github.com/naudio/NAudio>
- Microsoft Docs. (2022). Windows Forms Controls.
link: <https://docs.microsoft.com/en-us/dotnet/desktop/winforms>
- Microsoft Docs. (2022). Chart Controls for .NET Framework.
link: <https://docs.microsoft.com/en-us/dotnet/desktop/winforms/controls/chart>